Lecture 11

Threads

Real-world analogy: The sequential kitchen

- Task 1: Boil water (10 mins) → *Wait…*
- Task 2: Cook rice (15 mins) → *Wait…*
- Task 3: Slice vegetables (5 mins) → *Wait…*
- Task 4: Fry vegetables (10 mins).

➢ **Total Time: 40** Minutes.

This is how your Java programs have worked so far

The sequential code problem

```
System.out.println("1. Starting a file download...");
Thread.sleep(10000); // The program freezes here!
System.out.println("2. Download finished.");
System.out.println("3. Type your name: ");
Scanner scanner = new Scanner(System.in);
String name = scanner.nextLine();
```

- **Problem:** User couldn't type during download!
  - `Thread.sleep()` here simulates a long task
  - The file download is a **Blocking Operation**

Examples of blocking operations

- A website that freezes while an image loads.
- A video game that hangs completely every time it auto-saves your progress.
- A music player that pauses the song while you scroll the library.

☞ Procedural, single-tasking execution is terrible for interactive applications.

☞ We need a way to do tasks in the background.

The sequential code problem

```
public static void downloadFile(String fileName) {
    System.out.println("Starting download: " + fileName);
    try {
        // Simulating a 3-second download
        Thread.sleep(3000);
    } catch (InterruptedException e) {}
    System.out.println("Finished download: " + fileName);
}

public static void main(String[] args) {
    System.out.println("--- Starting Program ---");
    downloadFile("Movie_Part1.mp4"); // Blocks for 3 seconds
    downloadFile("Movie_Part2.mp4"); // Blocks for 3 seconds
    System.out.println("--- All Downloads Complete ---");
    // Total time: 6 seconds.
}
```

Real-world analogy: The **concurrent** kitchen

- Start at the same time:
  - Boiling water (10m)
  - Slicing vegetables (5m)
  - Cooking rice (15m)
- Fry vegetables (10m) right after slicing.

➢ All tasks are finished at 15 minutes.

➢ Total time: **15** minutes.

Multi-tasking as the solution

- We solve the kitchen problem by multitasking.
- We want our Java programs to do the same.
- We need a way to tell Java: "Start this task, and while it's working in the background, move on to the next line of code".

Processes vs. Threads

- **Process:** An executing program (e.g., your web browser, Spotify, your Java app).
  - A process has its own memory and resources.
  - Analogy: the Kitchen.
- **Thread:** A path of execution within a process.
  - Analogy: a Chef.
- **Key concept:** A kitchen (process) can have multiple chefs (threads) working at the same time, sharing the same ingredients (memory).
  - Up until now, your Java programs have had exactly one thread (the Main thread).

The "Main" thread

```
public static void main(String[] args) {
    Thread current = Thread.currentThread();
    System.out.println("Who is running this code? "
            + current.getName());
    // Output: Who is running this code? main
}
```

- `Thread.currentThread()` is used to get the currently running `Thread` object.
- `Thread.sleep()` is used to pause the current thread for a specified amount of time.

How to hire a new Chef?

- Java gives us two ways to create a new thread:
  - Extend the `Thread` class.
  - Implement the `Runnable` interface.
- The better practice is implementing `Runnable`.
- **Why?**
  - You can only extend one class, but you can implement many interfaces.
  - It keeps our OOP design flexible

Example: Extending Thread

```java
public class DownloadThread extends Thread {
    @Override
    public void run() {
        System.out.println("Download worker started...");
        // Do heavy work here
        try {
            Thread.sleep(10000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        System.out.println("Download completes.");
    }
}

// In main:
DownloadThread worker = new DownloadThread();
worker.start();
```

Example: Step 1 – Create a `Runnable`

```java
public class DownloadTask implements Runnable {
    private String fileName;

    public DownloadTask(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void run() {
        System.out.println("Starting download: " + fileName);
        try {
            Thread.sleep(3000); // Simulate 3 second download
        } catch (InterruptedException e) {}
        System.out.println("Finished download: " + fileName);
    }
}
```

- `run()` method contains the code you want the thread to execute.

Example: Step 2 – Start a Thread

```java
// Create some tasks
Runnable task1 = new DownloadTask("Movie_Part1.mp4");
Runnable task2 = new DownloadTask("Movie_Part2.mp4");

// Give the tasks to new Threads
Thread worker1 = new Thread(task1);
Thread worker2 = new Thread(task2);

// Start them! (DO NOT call run() directly)
worker1.start();
worker2.start();

System.out.println("Main method finished!");
```

What will be the output?

The output

```
--- Main method finished? ---
Starting download: Movie_Part1.mp4
Starting download: Movie_Part2.mp4
(Wait ~3 seconds)
Finished download: Movie_Part2.mp4
Finished download: Movie_Part1.mp4
```

- We have achieved multi-tasking!
- Why `"Main method finished!"` printed before the downloads?
  - The Main thread starts `worker1` and `worker2`, and then immediately moved to its last line of code (it doesn't wait for the threads to finish)

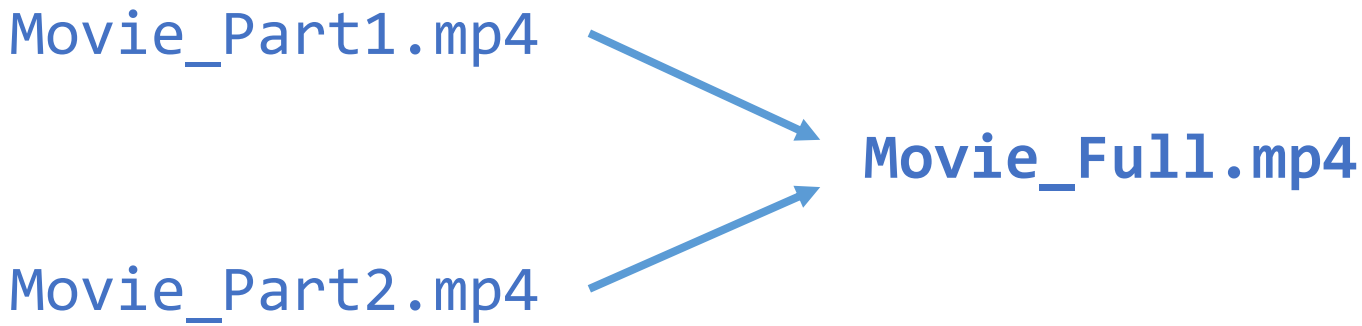
Beginner Trap: `start()` vs `run()`

```java
// WRONG WAY:
worker.run();
// Just a normal method call. The main thread does the work.

// RIGHT WAY:
worker.start();
// Spawns a new parallel timeline. The new thread does the work.
```

- `start()` is a complex, native Java method that talks to the OS, builds the new thread, and then the OS calls the `run()` method in the background.

BUT how to merge movie parts together?



- Your program needs to stitch the 2 movie parts together ***after*** downloading.
- If the Main Thread finishes instantly, **where** do we put the code to merge the movie parts?
  - Why can't we do it in the `run()` method of the `Runnable`?

The `join()` method

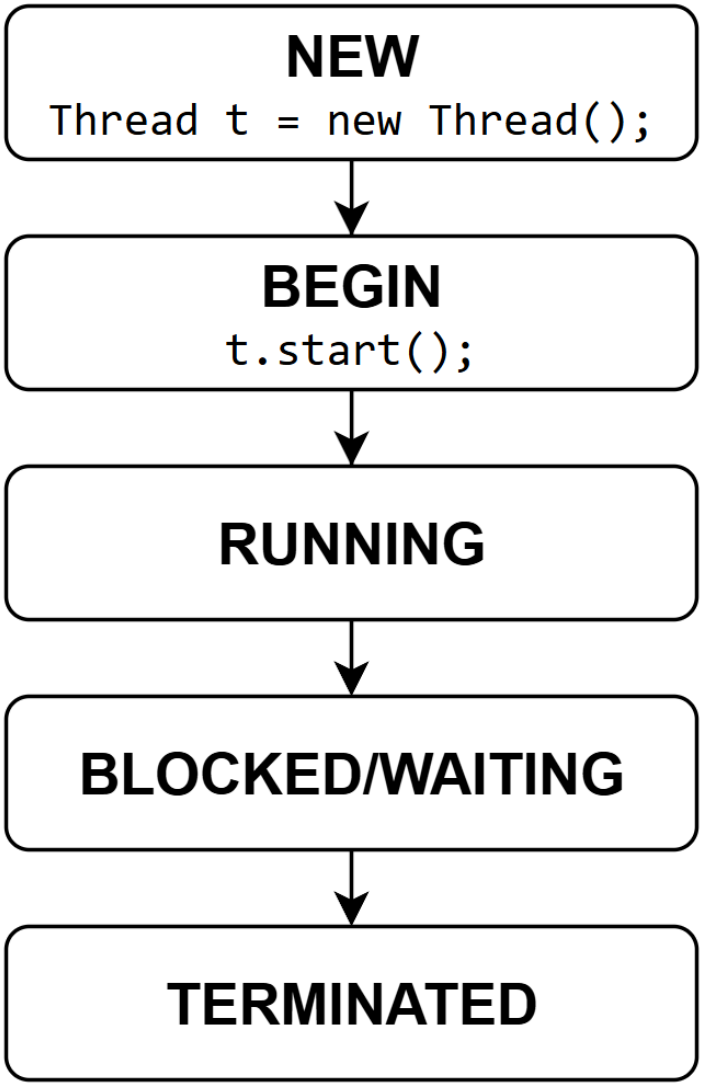
```
worker1.start();
worker2.start();

try {
    // Main thread pauses here and wait
    // for worker1 and worker2 to terminate
    worker1.join();
    worker2.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println("Downloads complete. Merging parts...");
```

- The Main thread will block at `worker1.join()` until `worker1` is completely done.

The Thread Lifecycle



Worker thread created but doesn't do anything yet

Worker thread is ready for work, waiting for a free CPU core

Thread is doing its work, the CPU is executing the code in the `run()` method

Thread is sleeping, waiting for input or waiting for other threads to terminate

`run()` method finishes

Fun Application Examples

That showcase how to use Threads

Example 1: Background data processing

Scenario:

- We have a massive CSV file (like a 10GB dataset or server logs).
- Parsing it takes 20 seconds.
- We want to show the user a loading spinner while it happens, instead of a frozen screen.

Example 1: Background data processing

```java
// The Main Thread handles the UI (Spinner)
public static void main(String[] args) throws Exception {
    Thread parserThread = new Thread(new DataParser());
    parserThread.start(); // Start heavy lifting in background

    // Main thread shows a spinner while the background thread is alive
    String[] spinner = {"|", "/", "-", "\\"};
    int i = 0;
    while (parserThread.isAlive()) {
        System.out.print("\rLoading Data " + spinner[i % spinner.length]);
        i++;
        Thread.sleep(100); // spinner speed
    }
    System.out.println("\nProcessing Complete!");
}
```

- `isAlive()` is used to check if a Thread is still running.

Example 2: The game loop

Scenario:

- Imagine a survival/base-building game where time constantly passes (crops grow, sun sets) even if the player doesn't press any buttons.
- While the time passes and the world is changing, the player can do his own things.

Example 2: The game loop

```java
// Thread 1: The Game World
class GameWorld implements Runnable {
    public void run() {
        while(true) {
            System.out.println("[World] 1 hour has passed." +
                    "Crops grew a bit.");
            try {
                Thread.sleep(5000);
                // 5 seconds = 1 in-game hour
            } catch (InterruptedException e){

            }
        }
    }
}
```

Example 2: The game loop

```
// Thread 2: Main Thread (Player Input)
public static void main(String[] args) {
    Thread world = new Thread(new GameWorld());
    world.start();

    Scanner scanner = new Scanner(System.in);
    while(true) {
        System.out.print("Enter command (attack, jump): ");
        String cmd = scanner.nextLine();
        System.out.println("Player executed: " + cmd);
    }
}
```

- The world continues to live on a 5-second timer, totally *independent* of how fast or slow the player types.

Lecture Summary

- The single-task problem
- Processes & Threads
- Creating & Starting Threads
- Coordinating multiple Threads
- Java Thread Life Cycle
- Example applications of multi-threading